Dossier de conception

>> Diginamic.fr

Révision	Rédacteurs	Date	Objet
1	R. BONNAMY	17/02/2021	Création du document
2	G. GUINET / D. CASTELLO / A. GARGOUA	26/08/2026	Adaptation du document au projet Hub événementiel
3	D. CASTELLO	23/10/2026	Mise à jour : tests, dépendances, sécurité

1. INTRODUCTION

1.1 Objet du document

Ce document a pour objectif de présenter l'essentiel des questions techniques liées à la mise en place de l'application de gestion d'un hub événementiel..

Ce document présente :

- Le diagramme de classes
- Le modèle physique de données
- Les règles de développement et la stratégie de tests
- Les règles de sécurité

SOMMAIRE

1.1.1 Table des matières

1 INTRODUCTION 2
1.1 Objet du document 2
2 SOMMAIRE 3
2.1.1 Table des matières 3
3 ARCHITECTURE LOGICIELLE 4
3.1 Produits et versions 4
3.1.1 Langages, frameworks et librairies spécifiques 4
3.1.2 Serveur de base de données 4
4 FOCUS TECHNIQUE 5
4.1 Diagramme de classes métier 5
4.2 Modèle physique de données 5
4.3 Règles de développement 5
5 TESTS ET INTÉGRATION 6
5.1 Stratégie de tests 6
5.2 Indicateurs de qualité de code 6

ARCHITECTURE LOGICIELLE

3.1 Produits et versions

3.1.1 Langages, frameworks et librairies spécifiques

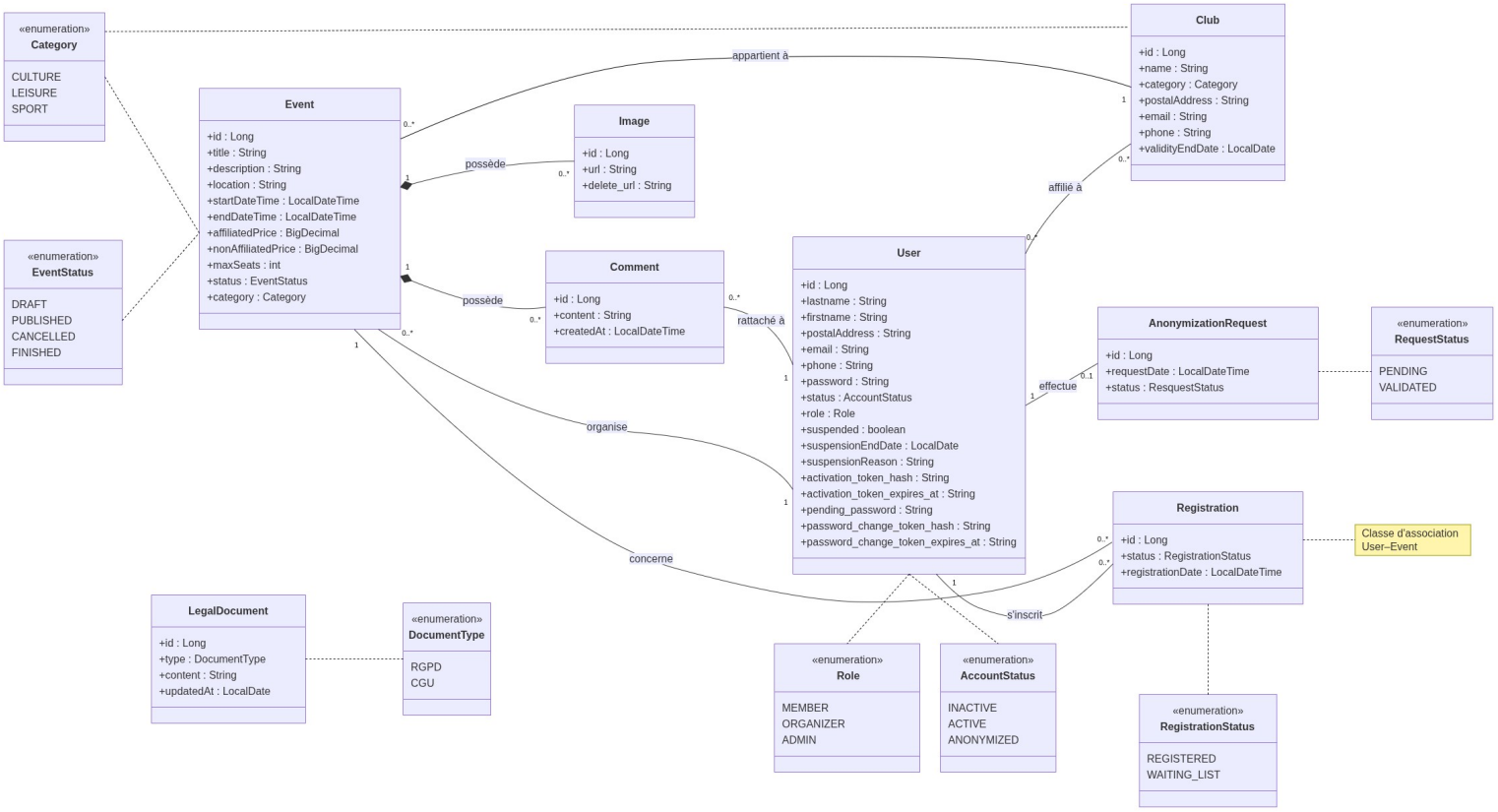
Nom	Version
Langage Java	21 (LTS)
Spring Boot	4.1.x
JPA (Hibernate)	3.2 (Hibernate 7.4)
Maven	3.9.x
Angular	22.2.x
iText (génération de PDF)	9.2.x
Apache POI (génération d'excel)	5.4.x
JUnit (Jupiter)	6.0.x
Mockito	5.23.x
Spring Security	7.1.x
JWT (jjwt)	0.12.x
Vitest (tests du front)	5.x
Node.js (build du front)	24

3.1.2 Serveur de base de données

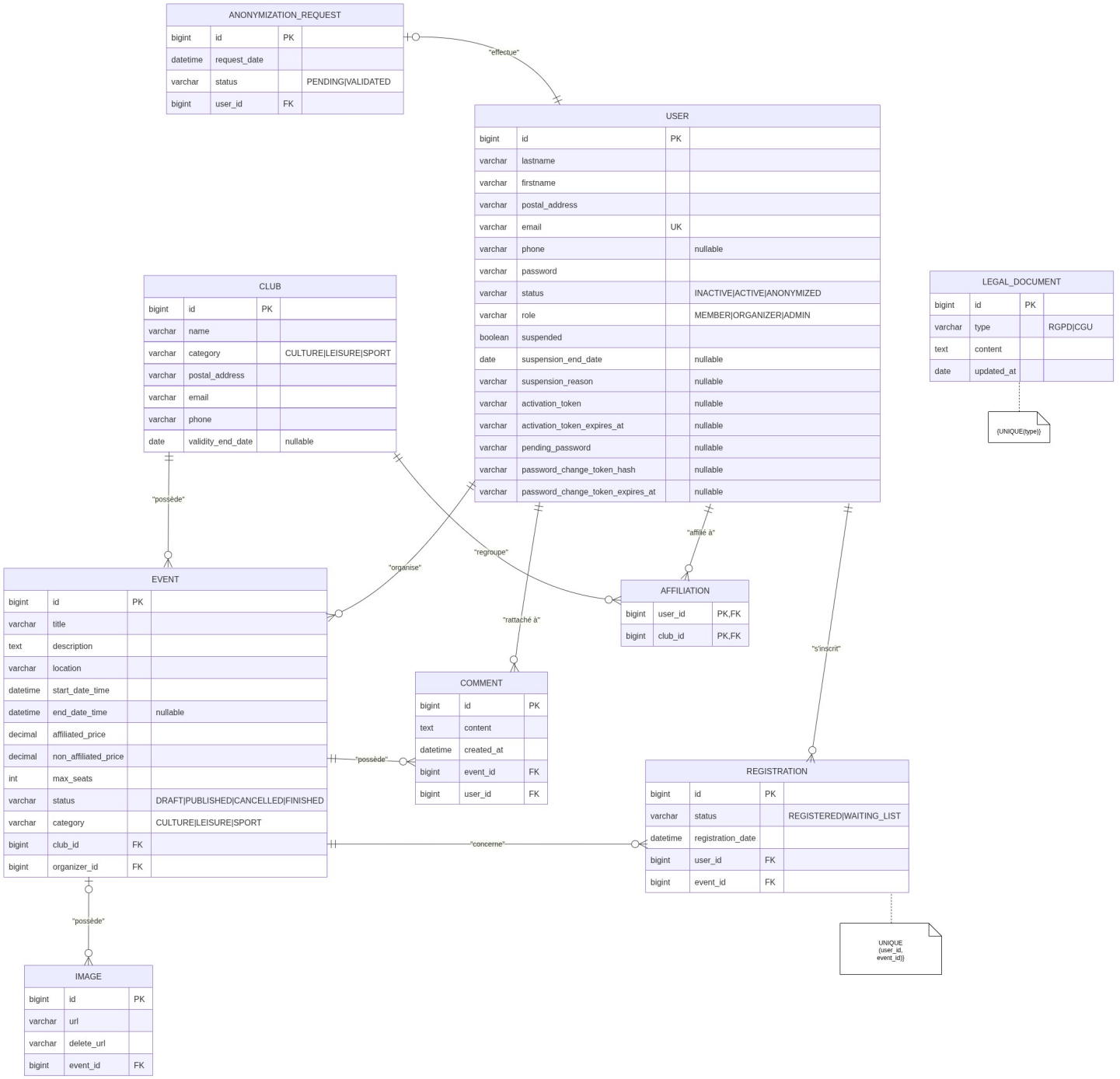
Nom	Version
MySQL	8.x

FOCUS TECHNIQUE

a. Diagramme de classes métier ou MCD



b. Diagramme entités relations ou MLD



c. Mise en place de la base de données

Le script de création des tables et des clés étrangères est dans le dépôt : backend/docs/database/schema.sql (MySQL 8). Il est appliqué à la base de production, dont le schéma est ensuite seulement vérifié par l'application (ddl-auto = validate).
En développement, Hibernate crée le schéma au démarrage et le fichier data.sql charge un jeu de données de démonstration (comptes, clubs, évènements, inscriptions). Les tests utilisent une base H2 en mémoire.

Règles de développement :

- Le code sera indenté de manière homogène.
- Les classes seront nommées en **PascalCase** et les méthodes et variables en **camelCase**.
- Les noms utilisés dans le code devront être explicites et représentatifs de leur rôle.
- Le code sera organisé en packages selon les différentes responsabilités de l'application.
- La duplication de code sera évitée autant que possible.
- Les méthodes devront rester simples et avoir une responsabilité clairement définie.
- Les commentaires seront utilisés lorsque le fonctionnement d'un traitement nécessite une explication particulière.

Le back-end sera organisé selon une architecture en couches :

Controller : réception des requêtes HTTP et gestion des réponses ;

DTO (Data Transfer Object) : objets utilisés pour transférer les données entre le back-end et le front-end, afin de ne pas exposer directement les entités de la base de données ;

Service : gestion de la logique métier et application des règles de gestion ;

Repository : accès et manipulation des données ;

Entity : représentation des données persistées en base de données.

Le front est une application Angular (composants autonomes et signaux), rangée en dossiers domain, core, shared et features. Le code est formaté avec Prettier et contrôlé par ESLint.

TESTS ET INTÉGRATION

a. Stratégie de tests

Le back est testé avec JUnit 6 et Mockito : environ 400 tests, lancés par ./mvnw test sur une base H2 en mémoire, sans service externe (l'envoi d'emails et l'hébergeur d'images sont simulés).

Les tests portent sur les services qui appliquent les règles de gestion : évènements, inscriptions et listes d'attente, affiliations, suspensions de comptes et demandes d'anonymisation.

Des tests d'intégration vérifient aussi les routes de l'API (codes HTTP, droits par rôle) et le bon enchaînement des couches.

Le front est testé avec Vitest : un fichier .spec.ts par composant ou par service (environ 75 fichiers), lancés par npx ng test.

Des tests dédiés vérifient la sécurité : droits d'accès par rôle, limitation du nombre de requêtes, contrôle des fichiers envoyés, absence de données sensibles dans les logs, HTML injecté affiché comme du texte.

b. Indicateurs de qualité de code

Intégration continue : à chaque Pull Request vers la branche dev, GitHub Actions lance pour le front ESLint, les tests et le build (.github/workflows/front-ci.yml). Le back est testé par ./mvnw test avant chaque Pull Request.
Contrôle des dépendances : OWASP Dependency-Check (back) et npm audit (front) recherchent les failles connues des bibliothèques. Les versions vulnérables ont été mises à jour ; une alerte restante, sur un outil de développement du front (vite), est acceptée.

c. Règles de sécurité

Les routes sont sécurisées par authentification et par rôle, et des règles précises s'appliquent aux mots de passe.
Sécurité des routes et des échanges :

- Authentification par jeton JWT signé, valable 1 heure, envoyé dans l'en-tête Authorization. L'API ne garde aucune session.
- Toutes les routes sont protégées par défaut. Seules quelques routes sont publiques : consultation des évènements, connexion, inscription, documents légaux.
- Chaque route sensible est réservée à un rôle (membre, organisateur, administrateur), et un organisateur ne peut modifier que ses propres évènements.
- Les liens reçus par email (activation du compte, changement de mot de passe) contiennent un jeton aléatoire, conservé haché en base, valable de 1 à 72 heures et utilisable une seule fois.
- Le nombre de requêtes est limité sur les routes sensibles (connexion, inscription, commentaires, envoi d'images) : l'API répond 429 au-delà du quota.
- Protection contre l'injection : requêtes paramétrées, affichage échappé par Angular, politique de sécurité du contenu (CSP) en production, fichiers envoyés vérifiés (type réel, 5 Mo, 20 fichiers par envoi).
- En production : documentation Swagger désactivée, messages d'erreur sans détail technique, en-têtes de sécurité HTTP, secrets dans des variables d'environnement.
- Les événements de sécurité (connexions refusées, accès interdits) sont journalisés, sans mot de passe, jeton ni adresse email complète.

Mots de passe : ils comportent au minimum 12 caractères, avec au moins une lettre majuscule, une lettre minuscule, un chiffre et un caractère spécial.

Les mots de passe ne sont jamais stockés en clair dans la base de données : ils sont hachés avec l'algorithme BCrypt.

Les recommandations de la CNIL relatives à la sécurité des mots de passe sont prises en compte.

Ces mesures répondent aux principaux risques de l'OWASP Top 10 : contrôle d'accès, authentification, injection, mauvaise configuration, composants vulnérables et journalisation.

Limites connues : le jeton de connexion est conservé dans le navigateur pendant son heure de validité, les quotas de requêtes sont gardés en mémoire du serveur et il n'y a pas d'authentification à deux facteurs.